

UNIVERSIDADE ESTADUAL DE SANTA CRUZ - UESC

**Escape Room Cooperativo: Documentação da Aplicação e do
Protocolo ERP/1.0**

Ilhéus - BA

2026

Descrição

Projeto da disciplina de Redes de Computadores realizado pelos alunos Arthur Araújo, Bruno Cardoso, João Pedro França e Lucas Vieira:

Um jogo de Escape Room cooperativo para 2 a 4 jogadores, jogado em terminal, com arquitetura cliente-servidor sobre sockets TCP e um protocolo de aplicação próprio, o **ERP/1.0** (*Escape Room Protocol, versão 1.0*).

Este documento cobre, nesta ordem: o propósito da aplicação, a motivação da escolha do protocolo de transporte, os requisitos mínimos de funcionamento, o funcionamento da aplicação (mecânica de jogo) e, por fim, a especificação completa do protocolo de aplicação: estados, eventos e todas as mensagens trocadas entre cliente e servidor.

Sumário

1. Propósito da aplicação
2. Arquitetura geral da solução
3. Motivação da escolha do protocolo de transporte
4. Requisitos mínimos de funcionamento
5. Funcionamento da aplicação
6. Protocolo de aplicação ERP/1.0
7. Sub-protocolo de descoberta (UDP broadcast)
8. Limitações conhecidas

1. Propósito da aplicação

A aplicação implementa um jogo de Escape Room cooperativo em rede local. Entre 2 e 4 jogadores entram na mesma partida, são distribuídos entre dois caminhos (papéis) diferentes do mapa e precisam resolver enigmas que dependem um do outro, trocando informações pelo chat embutido no próprio jogo, até alcançarem juntos a saída dentro de um tempo limite.

O objetivo didático do projeto é exercitar, na prática, os conceitos centrais da disciplina: comunicação cliente-servidor sobre sockets, definição de um protocolo de aplicação próprio (formato de mensagem, máquina de estados, tratamento de erros), concorrência (múltiplos clientes atendidos por múltiplas threads compartilhando um estado global protegido por lock) e tolerância a falhas de rede (quedas de conexão, desconexões no meio de uma partida).

O servidor é a única fonte de verdade do jogo: ele mantém o estado de cada jogador, das salas, dos objetos, do tempo restante e da condição de vitória ou derrota. Os clientes nunca alteram o estado diretamente, eles apenas enviam comandos e recebem, em troca, atualizações que o servidor decide enviar.

2. Arquitetura geral da solução

A solução é dividida em quatro módulos:

Arquivo	Responsabilidade
server.py	Servidor TCP. Aceita conexões, mantém a máquina de estados da partida, decide o que cada cliente recebe e quando.
client.py	Cliente de terminal. Envia comandos digitados pelo jogador e renderiza as mensagens recebidas do servidor.
launcher.py	Front-end de conveniência: cria o servidor (e entra como jogador local) ou descobre um servidor já existente na rede via broadcast UDP, evitando que o jogador precise digitar o IP manualmente.

game/protocol.py,	Núcleo do protocolo ERP/1.0 e da lógica de jogo: serialização
game/state.py,	de mensagens, máquina de estados da partida, comandos e
game/rooms.py	o mapa jogável.

O servidor é single-process e multi-thread: a thread principal apenas aceita conexões (accept) e delega cada conexão a uma thread dedicada (_handle_client), que fica bloqueada lendo linhas do socket daquele cliente. Como várias threads de cliente leem e escrevem o mesmo estado de jogo (GameState) e o mesmo dicionário de conexões, todo acesso a esses dados compartilhados é protegido por um threading.Lock (self.lock), garantindo que duas ações concorrentes (por exemplo, dois jogadores pegando o mesmo item ao mesmo tempo) sejam serializadas e não corrompam o estado do jogo.

3. Motivação da escolha do protocolo de transporte

O ERP/1.0, o protocolo principal do jogo, roda sobre **TCP**. Essa escolha foi feita por três motivos diretamente ligados às garantias que o TCP oferece e que o UDP não oferece nativamente:

Entrega garantida e ordenada. O jogo é uma máquina de estados compartilhada: um comando como *pegar chave_esquerda* só faz sentido se o servidor já processou, na ordem correta, os comandos anteriores daquele jogador (por exemplo, ter entrado na sala onde o item está). Se uma mensagem ACTION fosse perdida ou chegasse fora de ordem, algo possível em UDP, mas que o TCP evita por construção, o estado do servidor e a percepção do cliente sobre esse estado poderiam divergir de forma irrecuperável (um jogador pensaria que pegou um item que na verdade nunca chegou ao servidor, por exemplo). O mesmo vale para mensagens decisivas como GAME_OVER: perder esse pacote faria o cliente continuar esperando indefinidamente por um jogo que já terminou.

Modelo de conexão alinhado à arquitetura cliente-servidor. O TCP é orientado a conexão: cada jogador estabelece uma conexão persistente com o servidor enquanto está na partida, e o servidor já usa naturalmente essa conexão (o socket aceito) como identificador implícito de "quem está falando" antes mesmo do handshake de aplicação (JOIN) ser concluído. Isso elimina a necessidade de reimplementar, na camada de aplicação, controle de sessão, números de sequência e confirmações (ACKs) que o UDP exigiria para alcançar o mesmo nível de confiabilidade, esforço de engenharia que não traria benefício real para este jogo, que não tem requisitos de latência tão estritos quanto um jogo de ação em tempo real.

Delimitação de mensagens segura sobre um fluxo de bytes. O ERP/1.0 serializa cada mensagem como um objeto JSON seguido do caractere de quebra de linha (`\n`) como delimitador de quadro (*framing*). Isso só é seguro porque o `json.dumps` nunca emite um byte `0x0A` (`\n`) literal dentro de uma string do payload, uma quebra de linha presente em um texto de chat, por exemplo, é sempre escapada como `\n` (dois caracteres, barra e "n") dentro da string JSON. Logo, o delimitador de quadro nunca colide com o conteúdo da mensagem. Essa estratégia depende de receber os bytes na mesma ordem em que foram enviados, o que o TCP garante e o UDP não garante.

Por essas razões, o UDP foi reservado apenas para a funcionalidade auxiliar de descoberta automática de servidor na rede local (detalhada na seção 7), onde a perda ocasional de pacotes é inofensiva, a mensagem de presença é simplesmente reenviada a cada poucos segundos, e a necessidade de fazer broadcast para um endereço ainda desconhecido é algo que o TCP, por ser orientado a conexão ponto-a-ponto, não permite fazer diretamente.

4. Requisitos mínimos de funcionamento

- **Python 3.10 ou superior** em todas as máquinas (servidor e clientes). O código usa a sintaxe de União de tipos do PEP 604 (`str | None`) diretamente nas assinaturas de função, sem `from __future__ import annotations`, o que exige Python 3.10+ em tempo de execução.
- **Rede TCP/IP** entre as máquinas dos jogadores. Se todos jogarem na mesma máquina, basta usar `127.0.0.1`. Se jogarem em máquinas diferentes, elas precisam estar na mesma rede (ou ter rota TCP válida entre si) e a porta TCP do jogo (5000 por padrão) precisa estar liberada no firewall do host do servidor.
- **Mesma sub-rede local (LAN)**, apenas se for usado o recurso de descoberta automática do `launcher.py`: pacotes de broadcast UDP não atravessam roteadores por padrão, então a auto-descoberta só funciona entre máquinas no mesmo segmento de rede. Fora desse caso, o modo manual (`client.py --host <IP>`) funciona em qualquer topologia onde a porta TCP seja alcançável.
- **Porta UDP 5001 liberada**, apenas se for usado o `launcher.py` (descoberta automática). Não é necessária se o servidor e os clientes forem iniciados manualmente via `server.py` e `client.py --host`.
- **Terminal com suporte a UTF-8 e a sequências de escape ANSI**, já que o cliente usa caracteres Unicode (emojis, bordas de caixa) e códigos de escape para reescrever a linha de prompt sem apagar o que o jogador está digitando.
- **Módulo readline** disponível no Python do cliente (`client.py` importa `readline` diretamente). Em Linux e macOS isso já vem com o interpretador padrão; em

Windows nativo pode ser necessário um pacote equivalente (como pyreadline3), ou executar o cliente em um ambiente POSIX (WSL, por exemplo).

- **Entre 2 e 4 jogadores** por partida — esse intervalo é uma regra do próprio protocolo, não apenas uma recomendação (ver MIN_PLAYERS/MAX_PLAYERS na seção 6).

5. Funcionamento da aplicação

5.1. Papéis, mapa e cooperação

O mapa atual, "Hospital Abandonado", tem dois caminhos essenciais e independentes, role 0 (caminho da Recepção) e role 1 (caminho da Sala de Força), que convergem no Corredor Central. Cada jogador é automaticamente associado a um desses papéis no momento em que entra na partida, e os papéis são redistribuídos de forma alternada conforme o número de jogadores conectados (2 jogadores → 0,1; 3 jogadores → 0,1,0; 4 jogadores → 0,1,0,1).

Cada caminho contém pistas que o jogador do *outro* caminho precisa para avançar (por exemplo, a voltagem do gerador, escrita em uma placa na Sala de Força, é necessária para ligar um terminal na Recepção). O jogo não anuncia automaticamente cada item encontrado, a cooperação via comando *chat* `<mensagem>` é, portanto, parte essencial da jogabilidade, e não apenas um recurso social. Alguns eventos cooperativos mais críticos (energia religada, porta destrancada remotamente, reencontro da equipe no corredor) são, esses sim, anunciados automaticamente pelo servidor a todos os jogadores via `PLAYER_EVENT`, para que ninguém perca uma virada de jogo importante mesmo sem prestar atenção ao chat.

A saída final do Corredor Central exige duas chaves físicas distintas (uma de cada caminho), cada uma acionada em um dispositivo correspondente, uma dependência deliberadamente desenhada para forçar que ambos os jogadores estejam presentes e tenham concluído seus respectivos caminhos antes de qualquer um conseguir escapar.

5.2. Comandos do jogador

ready	sair	chat <mensagem>
olhar / sala	inventario	dica

votar <mapa>

examinar <objeto> pegar <objeto>

usar <item> em <objeto> colocar <senha> no <objeto>

ir <norte | sul | leste | oeste>

O parser de comandos normaliza acentos e maiúsculas/minúsculas antes de interpretar qualquer entrada (usando decomposição Unicode NFD para remover marcas diacríticas), de forma que "Inventário", "inventario" e "INVENTARIO" sejam tratados de forma idêntica. Além disso, em salas que possuem apenas uma porta interativa visível, o jogo aceita o alvo genérico porta em vez do nome específico (por exemplo, colocar 1968 na porta funciona na Sala de Força mesmo que o objeto interno se chame porta_sul), resolvendo a ambiguidade pelo contexto da sala em que o jogador está.

5.3. Robustez contra desconexões (anti-softlock)

Como o mapa depende de ambos os papéis essenciais estarem presentes, o servidor trata desconexões de forma diferente de acordo com o momento da partida:

- **No lobby**, se alguém sai, os papéis dos jogadores restantes são redistribuídos imediatamente e o estado de "pronto" de todos é limpo, para impedir que a sala fique, por exemplo, só com jogadores no mesmo papel.
- **Durante a contagem regressiva**, se o número de jogadores cair abaixo do mínimo ou um papel essencial ficar descoberto, a contagem é cancelada, o mapa é resetado e todos voltam ao lobby precisando confirmar **ready** de novo.
- **Durante a partida**, se restarem menos de 2 jogadores conectados, o jogo termina em derrota. Se restarem 2 ou mais jogadores, mas um papel essencial ficar sem ninguém, a partida inteira é reiniciada automaticamente (mapa resetado, inventários limpos, papéis redistribuídos entre quem ficou). Se a partida pode continuar normalmente, os itens que estavam no inventário do jogador que caiu são devolvidos ao chão da sala onde ele estava, para que outro jogador do mesmo caminho possa recuperá-los sem precisar refazer os enigmas já resolvidos.

Essa lógica evita um *softlock*: uma partida que continua rodando (ou começa) sem que seja matematicamente possível terminá-la, ou um item essencial que desaparece para sempre junto com a conexão de quem o carregava.

6. Protocolo de aplicação ERP/1.0

6.1. Formato geral das mensagens

Toda mensagem ERP/1.0 é um único objeto JSON, codificado em UTF-8, com exatamente dois campos obrigatórios — `type` (string) e `payload` (objeto) — seguido do caractere `\n` como delimitador de quadro:

```
{"type": "ACTION", "payload": {"command": "examinar mesa"}}
```

A camada de transporte (TCP) entrega um fluxo contínuo de bytes sem preservar fronteiras de mensagem; por isso, tanto o servidor (`_readlines`) quanto o cliente (`receive_loop`) mantêm um *buffer* próprio: a cada `recv()` (em blocos de até 4096 bytes), os bytes recebidos são acumulados nesse buffer e, a cada vez que um `\n` aparece, tudo o que vem antes dele é decodificado como uma mensagem completa e o restante permanece no buffer para a próxima iteração. Esse mecanismo é o que permite reconstruir corretamente os limites de mensagem mesmo quando o TCP entrega um payload fragmentado em múltiplos pacotes, ou múltiplas mensagens concatenadas em uma única chamada de `recv()`.

Se a linha recebida não for um JSON válido ou não tiver os campos `type/payload`, o servidor responde com `ERROR [INVALID_ACTION]` e descarta a linha, sem encerrar a conexão.

6.2. Máquina de estados do servidor

O servidor (e, por extensão, cada partida) está sempre em um destes quatro estados, definidos em `game/protocol.py::State`:

Estado	Significado
WAITING_PL AYERS	Lobby. Jogadores podem entrar (JOIN), votar no mapa (MAP_VOTE) e confirmar presença (READY).
COUNTDOWN N	Contagem regressiva de 10 segundos antes do início, ativada quando todos os jogadores confirmaram READY e todos os papéis essenciais estão preenchidos.

IN_GAME Partida em andamento. Jogadores enviam ACTION e CHAT; o servidor controla o cronômetro da partida.

GAME_OVER Partida terminada (vitória ou derrota). Estado transitório: após 10 segundos, o servidor reseta automaticamente o jogo e volta para WAITING_PLAYERS.

stateDiagram-v2

[*] --> WAITING_PLAYERS

WAITING_PLAYERS --> COUNTDOWN: todos enviaram READY \n e papéis essenciais completos

COUNTDOWN --> WAITING_PLAYERS: jogador insuficiente ou \n papel essencial ausente (cancelado)

COUNTDOWN --> IN_GAME: contagem chegou a 0

IN_GAME --> WAITING_PLAYERS: papel essencial perdido \n por desconexão (reset automático)

IN_GAME --> GAME_OVER: tempo esgotado (lose) \n ou saída alcançada (win)

GAME_OVER --> WAITING_PLAYERS: reset automático \n após 10 segundos

Note que a transição de IN_GAME direto para WAITING_PLAYERS (sem passar por GAME_OVER) é deliberada: quando a partida é reiniciada por perda de um papel essencial, não houve nem vitória nem derrota, apenas uma condição que tornaria o jogo impossível de terminar, então ele simplesmente recomeça.

6.3. Mensagens Cliente → Servidor

Mensagem	Payload	Estado(s) em que é válida	Efeito
JOIN	{"username": str}	WAITING_PLAYER S, e apenas antes de a conexão já ter um player_id atribuído	Tenta registrar o jogador na partida. Em caso de sucesso, o servidor responde WELCOME; em caso de falha, responde ERROR e a conexão pode tentar JOIN novamente.
READY	{}	WAITING_PLAYER S (ignorada silenciosamente em outros estados)	Marca o jogador como pronto. Se todos os jogadores ficarem prontos e os papéis essenciais estiverem completos, inicia a COUNTDOWN.
MAP_VOTE	{"map": str}	WAITING_PLAYER S (ignorada em outros estados)	Registra (ou substitui) o voto do jogador em um mapa. Mapas inválidos geram ERROR [INVALID_ACTION].
ACTION	{"command": str}	IN_GAME (gera ERROR [NOT_IN_GAME] em qualquer outro estado)	Comando de jogo em linguagem natural (ex.: pegar chave, ir norte), processado pelo motor de jogo (GameState.process_action).
CHAT	{"message": str}	Qualquer estado, desde que o jogador já tenha um player_id	Mensagem de texto livre, redistribuída a todos os jogadores conectados.

DISCONNECT	{}	Qualquer estado	Saída voluntária e explícita. Tem o mesmo efeito de tratamento que uma queda de conexão abrupta (seção 5.3), mas é iniciada pelo próprio cliente em vez de detectada por uma exceção de socket.
------------	----	-----------------	---

Toda mensagem cujo type não esteja nessa lista recebe ERROR [INVALID_ACTION]. Qualquer mensagem que não seja JOIN enviada antes do handshake (isto é, antes de a conexão ter um player_id válido) também recebe ERROR [INVALID_ACTION], com a mensagem "Envie JOIN antes de qualquer outro comando."

6.4. Mensagens Servidor → Cliente

Mensagem	Payload	Quando é enviada	Destinatário
WELCOME	{"player_id": str, "server_state": str}	Resposta direta a um JOIN aceito	Unicast (só para quem entrou)
MAP_VOTE_STATE	{"votes": {mapa: contagem}, "maps": {mapa: nome}}	A cada novo JOIN aceito (estado inicial) e a cada MAP_VOTE recebido	Unicast no primeiro caso; broadcast no segundo
MAP_SELECTED	{"map": str, "map_name": str}	Ao final da COUNTDOWN, quando o mapa vencedor da votação é definido	Broadcast

LOBBY_UPDATE	{ "players": [{"player_id", "username", "ready"}], "ready_count": int, "total": int }	A cada JOIN, READY ou desconexão enquanto a partida está no lobby	Broadcast
COUNTDOWN	{ "seconds": int }	A cada segundo durante a contagem regressiva (10 → 1)	Broadcast
GAME_START	{ "room": str, "description": str, "time_limit": int }	Uma vez, ao transitar de COUNTDOWN para IN_GAME	Unicast individual: cada jogador recebe sua própria sala inicial, que depende do papel (caminho assimétrico)
ACTION_RESULT	{ "success": bool, "message": str, "state_changed": bool }	Resposta a um ACTION que não altera o que outros jogadores veem (mensagens de erro, descrições, inventário, etc.)	Unicast (só para quem enviou o comando)

ROOM_UPDATE	<pre> {"room_state": {"name","exits"}, "objects": [str], "players_here": [str]} </pre>	Sempre que o estado visível de uma sala muda (item pego, porta destrancada, jogador entrou/saiu da sala)	Broadcast a todos os jogadores conectados (o cliente filtra localmente e só renderiza se ele próprio estiver players_here)
PLAYER_EVENT	<pre> {"event": str, "player": str, "detail": str} </pre>	Eventos notáveis: joined, left, moved (reencontro no corredor), solved (enigma cooperativo resolvido remotamente), countdown_cancelled, match_reset	Broadcast
CHAT_BROADCAST	<pre> {"from": str, "message": str} </pre>	Resposta a um CHAT	Broadcast
TIMER_UPDATE	<pre> {"remaining": int} </pre>	Durante IN_GAME, a cada 120 segundos (2 minutos) ou nos marcos críticos de 300, 60, 30 e 10 segundos restantes	Broadcast

HINT	<code>{"text": str}</code>	Resposta ao comando dica	Unicast (só para quem pediu)
GAME_OVER	<code>{"result": "win" "lose", "time_elapsed": int, "message": str}</code>	Quando o tempo esgota, quando alguém atravessa a saída final, ou quando restam menos de 2 jogadores durante a partida	Broadcast
ERROR	<code>{"code": str, "message": str}</code>	Qualquer rejeição de mensagem (ver tabela de códigos de erro, seção 6.5)	Unicast (só para quem causou o erro)

Um detalhe de roteamento importante: quando um ACTION é processado, o motor de jogo pode gerar várias mensagens de resposta de uma só vez (por exemplo, um ACTION_RESULT mais um ROOM_UPDATE). O servidor decide o destinatário **por tipo de mensagem**, não por uma flag explícita: mensagens do tipo ROOM_UPDATE e PLAYER_EVENT são sempre transmitidas a todos os jogadores conectados (porque alteram algo que outros jogadores podem ver ou precisam saber), enquanto qualquer outro tipo gerado a partir de uma ação (ACTION_RESULT, HINT) é enviado apenas a quem executou o comando.

A condição de vitória não tem um tipo de mensagem próprio: internamente, um comando ir <direção> que leva à saída final gera um ACTION_RESULT cujo campo message é literalmente a string sentinela "__ESCAPED__". O servidor inspeciona as respostas geradas e, ao encontrar essa sentinela, dispara o GAME_OVER de vitória para toda a equipe; o cliente, por sua vez, sabe ignorar essa mensagem sentinela na renderização (ela nunca é mostrada como texto ao jogador, que só vê o GAME_OVER subsequente).

6.5. Códigos de erro

Código	Gerado quando
NAME_TAKEN	JOIN com um nome de usuário já em uso por outro jogador conectado
GAME_FULL	JOIN quando a partida já tem 4 jogadores conectados
GAME_IN_PROGRESS	JOIN quando o servidor não está em WAITING_PLAYERS (contagem regressiva ou partida em andamento)
INVALID_ACTION	JOIN com nome vazio; MAP_VOTE com mapa inexistente; tipo de mensagem desconhecido; mensagem mal formada (JSON inválido ou sem type/payload); qualquer mensagem enviada antes do JOIN ser concluído
NOT_IN_GAME	ACTION enviada quando o estado do servidor não é IN_GAME

6.6. Fluxo completo de uma partida

sequenceDiagram

participant A as Cliente A

participant B as Cliente B

participant S as Servidor

A->>S: JOIN {username: "Ana"}

S->>A: WELCOME {player_id, server_state}

S->>A: MAP_VOTE_STATE

S-->>A: LOBBY_UPDATE (broadcast)

S-->>A: PLAYER_EVENT "joined" (broadcast)

B->>S: JOIN {username: "Bruno"}

S->>B: WELCOME {player_id, server_state}

S->>B: MAP_VOTE_STATE

S-->>A: LOBBY_UPDATE (broadcast)

S-->>B: PLAYER_EVENT "joined" (broadcast)

A->>S: MAP_VOTE {map: "hospital"}

S-->>A: MAP_VOTE_STATE (broadcast)

A->>S: READY

B->>S: READY

S-->>A: LOBBY_UPDATE (broadcast)

Note over S: all_ready() == true

loop 10 segundos

 S-->>A: COUNTDOWN {seconds} (broadcast)

end

S-->>A: MAP_SELECTED (broadcast)

S->>A: GAME_START {room: "recepcao"} (unicast)

S->>B: GAME_START {room: "sala_de_forca"} (unicast)

loop Durante a partida

A->>S: ACTION {command: "examinar terminal"}

S->>A: ACTION_RESULT

A->>S: CHAT {message: "achei a senha 1968"}

S-->>A: CHAT_BROADCAST (broadcast)

Note over S: a cada 120s ou em marcos críticos

S-->>A: TIMER_UPDATE (broadcast)

end

A->>S: ACTION {command: "ir norte"}

Note over S: destino é a saída final

S-->>A: GAME_OVER {result: "win"} (broadcast)

Note over S: após 10 segundos, reset automático

S-->>A: LOBBY_UPDATE (broadcast)

6.7. Concorrência e consistência

Cada cliente é atendido por uma thread dedicada no servidor, mas todas as threads compartilham a mesma instância de GameState e o mesmo dicionário de conexões. Toda leitura ou escrita nesse estado compartilhado — registrar um JOIN, processar um ACTION, tratar uma desconexão — é feita dentro de um único threading.Lock (self.lock). Isso significa que, do ponto de vista do protocolo, o servidor processa uma mensagem de cada vez de forma atômica em relação ao estado do jogo: não existe uma janela de tempo em que duas ações concorrentes (por exemplo, dois jogadores tentando pegar o mesmo item no mesmo instante) possam ler o mesmo estado "antigo" e gerar um resultado inconsistente. O custo dessa escolha é que uma operação longa dentro do lock bloquearia as demais threads, mas como cada handler é puramente CPU-bound e rápido (sem I/O bloqueante dentro do lock), esse custo é desprezível na prática.

7. Sub-protocolo de descoberta (UDP broadcast)

O launcher.py implementa, à parte do ERP/1.0, um protocolo auxiliar minimalista sobre UDP para que um jogador não precise descobrir e digitar manualmente o endereço IP do servidor. Esse sub-protocolo usa a porta UDP **5001** (distinta da porta TCP **5000** do jogo) e uma única mensagem de texto simples, sem JSON:

```
ESCAPE_ROOM_ERP1:<ip_do_servidor>:<porta_tcp_do_jogo>
```

Quando alguém escolhe "Criar partida", além de iniciar o servidor TCP, o launcher.py passa a transmitir essa mensagem como broadcast UDP (endereço <broadcast>, porta 5001) a cada 2 segundos, indefinidamente, enquanto o servidor estiver no ar. Quando alguém escolhe "Entrar em partida", o cliente abre um socket UDP, faz bind na porta 5001 e escuta por até 6 segundos; ao receber uma mensagem que comece com o prefixo ESCAPE_ROOM_ERP1:, extrai o IP e a porta dela e os usa para abrir a conexão TCP real do jogo. Se nenhuma mensagem chegar dentro do tempo limite, o cliente cai no modo manual e pede o IP por teclado.

Esse sub-protocolo é deliberadamente tolerante a perdas: como a mensagem de presença é reenviada a cada 2 segundos, perder um pacote isoladamente não tem consequência, o próximo broadcast chega pouco depois. Por isso, UDP (sem qualquer garantia de entrega) é suficiente e, de fato, preferível ao TCP aqui: a única forma de alcançar uma máquina cujo endereço ainda não se conhece, em uma LAN, é por broadcast, e broadcast não é uma operação válida sobre um socket TCP (que é ponto-a-ponto e orientado a conexão).

8. Limitações conhecidas

O protocolo atualmente sustenta apenas um mapa jogável (hospital), embora a infraestrutura de votação (MAP_VOTE/MAP_VOTE_STATE/MAP_SELECTED) já tenha sido desenhada de forma genérica para suportar múltiplos mapas no dicionário MAPS, caso outros sejam adicionados no futuro.

O handler de JOIN não verifica explicitamente se a conexão já possui um player_id atribuído antes de processar um novo JOIN. Na prática isso não chega a ser um problema com o cliente de referência (client.py), que só envia JOIN repetidamente durante o fluxo de "nome já em uso" — justamente o caso em que o player_id ainda é None, pois um JOIN rejeitado nunca registra o jogador. Ainda assim, do ponto de vista da especificação do protocolo, um cliente mal-comportado que enviasse um segundo JOIN válido na mesma conexão já autenticada poderia registrar um jogador "fantasma" extra compartilhando o mesmo socket, sem que esse caso seja explicitamente rejeitado pelo servidor.

A auto-descoberta via UDP broadcast só funciona dentro do mesmo segmento de rede local, por ser uma limitação intrínseca de pacotes de broadcast IP (eles não são roteados). Em redes com múltiplas sub-redes, VPNs, ou isolamento de cliente Wi-Fi (comum em redes públicas/corporativas), os jogadores precisam recorrer ao modo manual informando o IP do servidor diretamente.